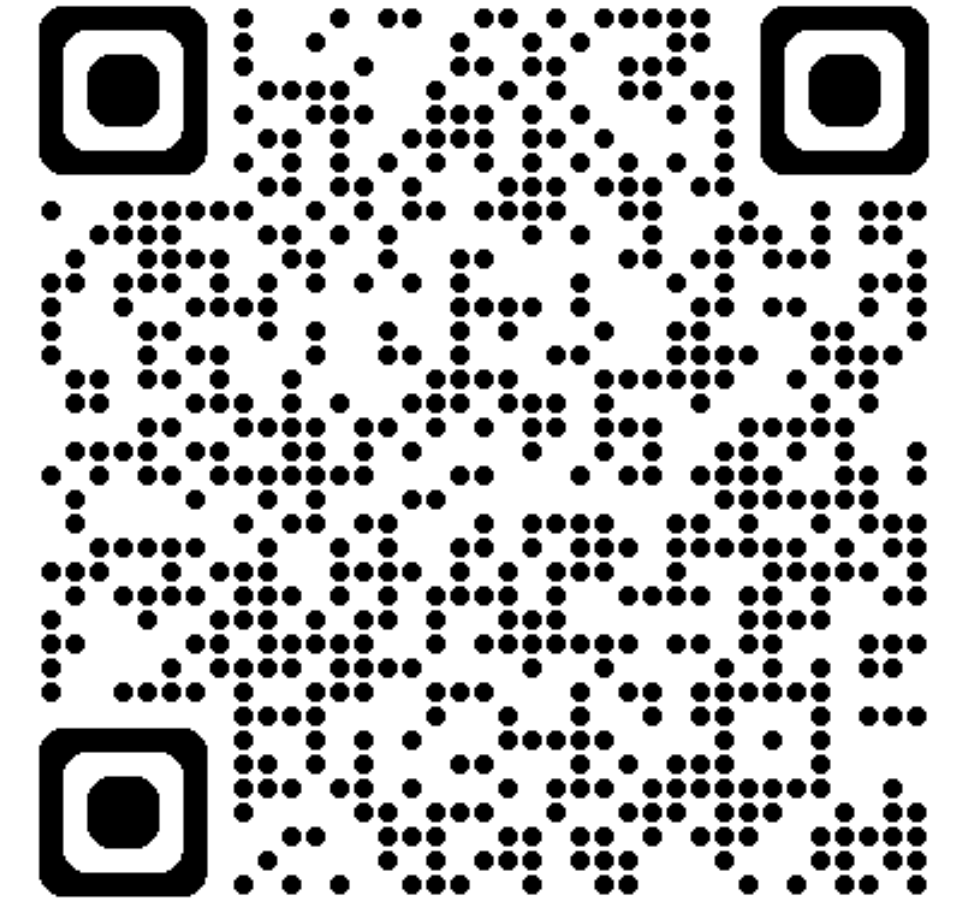


Übungsstunde 8 - Welcome back



Heute:

- Rückblick & Repetitions Quiz
- Binary Search Trees
- Heaps

CodeExpert Feedback kommt!

Leider noch nicht dazu gekommen, am Wochenende nehme ich mir Zeit!

& danke für tolles Feedback!

Repetition Mastertheorem.

□ Master-Theorem – Kochrezept

$$T(n) = a \cdot T(n/b) + f(n)$$

$$a \geq 1, b > 1$$

1. $k = \log_b a$

2. Vergleiche $f(n)$ mit n^k :

Fall 1: $f(n) = O(n^{(k - \varepsilon)})$

$$\rightarrow T(n) = \Theta(n^k)$$

Fall 2: $f(n) = \Theta(n^k \cdot \log^p n)$

$$\rightarrow T(n) = \Theta(n^k \cdot \log^{(p+1)} n)$$

Fall 3: $f(n) = \Omega(n^{(k + \varepsilon)})$, Regularity erfüllt

$$\rightarrow T(n) = \Theta(f(n))$$

```
def case1(n):  
    if n <= 1: return  
  
    # f(n) = 1 (konstante Arbeit, z.B. eine Zuweisung)  
    x = 0  
  
    # a=4 rekursive Aufrufe mit b=2 (Problemhalbierung)  
    for _ in range(4):  
        case1(n // 2)
```


Repetition Mastertheorem.

□ Master-Theorem – Kochrezept

$$T(n) = a \cdot T(n/b) + f(n)$$

$$a \geq 1, b > 1$$

1. $k = \log_b a$

2. Vergleiche $f(n)$ mit n^k :

Fall 1: $f(n) = O(n^{(k - \varepsilon)})$

$$\rightarrow T(n) = \Theta(n^k)$$

Fall 2: $f(n) = \Theta(n^k \cdot \log^p n)$

$$\rightarrow T(n) = \Theta(n^k \cdot \log^{(p+1)} n)$$

Fall 3: $f(n) = \Omega(n^{(k + \varepsilon)})$, Regularity erfüllt

$$\rightarrow T(n) = \Theta(f(n))$$

```
def case1(n):  
    if n <= 1: return  
  
    # f(n) = 1 (konstante Arbeit, z.B. eine Zuweisung)  
    x = 0  
  
    # a=4 rekursive Aufrufe mit b=2 (Problemhalbierung)  
    for _ in range(4):  
        case1(n // 2)
```

$a = 4$: Vierfache Verzweigung pro Ebene.

$b = 2$: Das Problem schrumpft pro Schritt um die Hälfte.

$k = \log_2 4 = 2$: Das "kritische" Wachstum der Rekursion ist quadratisch (n^2).

epsilon = 2 ; daher grösser als 0 , fall 1, $\Rightarrow O(n^2)$

Aliasing refresher

```
def transform_data(data):  
    data['status'] = 'processed'  
    data = {'status': 'new', 'value': 42}  
    data['value'] += 1  
    return data  
  
info = {'status': 'pending', 'value': 10}  
result = transform_data(info)  
  
# Was ist der Wert von info?  
print(info)
```

Aliasing refresher

```
def transform_data(data):  
    data['status'] = 'processed'  
    data = {'status': 'new', 'value': 42}  
    data['value'] += 1  
    return data
```

```
info = {'status': 'pending', 'value': 10}  
result = transform_data(info)
```

```
# Was ist der Wert von info?  
print(info)
```

```
{'status': 'processed', 'value': 10}
```

ab hier verlieren wir Zugriff auf info.

OOP refresher

gegeben:

```
class Waggon:
    def __init__(self, id: int):
        self.id = id
        self.naechster: Optional["Waggon"] = None

class Zug:
    def __init__(self):
        self.erster_waggon: Optional[Waggon] = None

    def ankoppeln(self, neue_id: int):
        neuer_waggon = Waggon(neue_id)

        # Fall 1: Der Zug ist noch leer
        if self.erster_waggon is None:
            self.erster_waggon = neuer_waggon
            return

        # Fall 2: Den letzten Waggon suchen (Traversierung)
```

Aufgabe: implementiere den (pseudo) code um den wagen am ende anzukoppeln

tipp: Woran können wir erkennen, dass ein Waggon der letzte ist?

OOP refresher

Lösung:

```
class Waggon:
    def __init__(self, id: int):
        self.id = id
        self.naechster: Optional["Waggon"] = None

class Zug:
    def __init__(self):
        self.erster_waggon: Optional[Waggon] = None

    def ankoppeln(self, neue_id: int):
        neuer_waggon = Waggon(neue_id)

        # Fall 1: Der Zug ist noch leer
        if self.erster_waggon is None:
            self.erster_waggon = neuer_waggon
            return

        # Fall 2: Den letzten Waggon suchen (Traversierung)
        aktuell = self.erster_waggon
        while aktuell.naechster is not None:
            aktuell = aktuell.naechster

        # Den neuen Waggon am Ende anfügen
        aktuell.naechster = neuer_waggon
```


Bäume

Ein *Baum* ist

- Verallgemeinerte Liste: Knoten können mehrere Nachfolger haben.
- Spezieller Graph: Graphen bestehen aus Knoten und Kanten. Ein Baum ist ein zusammenhängender, gerichteter, azyklischer Graph.

Binäre Bäume

Ein *binärer Baum* ist

- entweder ein leerer Baum, also ein Blatt,
- oder ein Knoten mit zwei Bin. Bäumen T_l (linker Teilbaum) und T_r (rechter Teilbaum) als linken und rechten Nachfolger.
 - Einen Knoten, der nur leere Nachfolger hat, bezeichnen wir als Blatt-Knoten, nicht zu verwechseln mit einem Blatt.

In jedem Knoten v wird gespeichert:

- ein Schlüssel $v.\mathbf{key}$ und
- zwei Zeiger $v.\mathbf{left}$ und $v.\mathbf{right}$ auf die Wurzeln der linken und rechten Teilbäume.

Ein Blatt wird durch den null-Zeiger repräsentiert. Um überfüllte Diagramme zu vermeiden, lassen wir beim Zeichnen von Bäumen manchmal die Kanten zu den Blättern weg.

Arten von Binären Bäumen

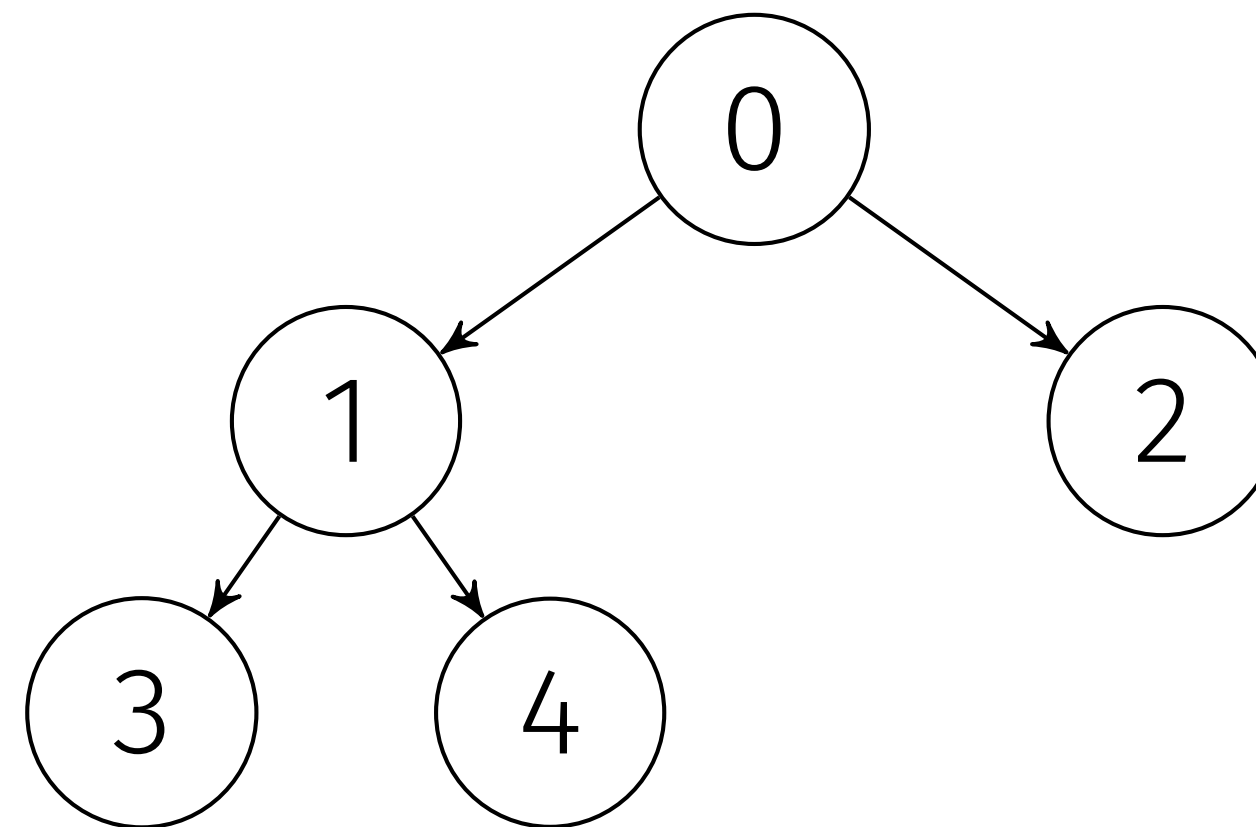
Es gibt drei Arten von Binärbäumen basierend auf der **Anzahl der Kinder**:

- Voller Binärbaum
- Vollständiger Binärbaum
- Perfekter Binärbaum

Hinweis: In der Literatur existieren verschiedene Bezeichnungen und Definitionen!

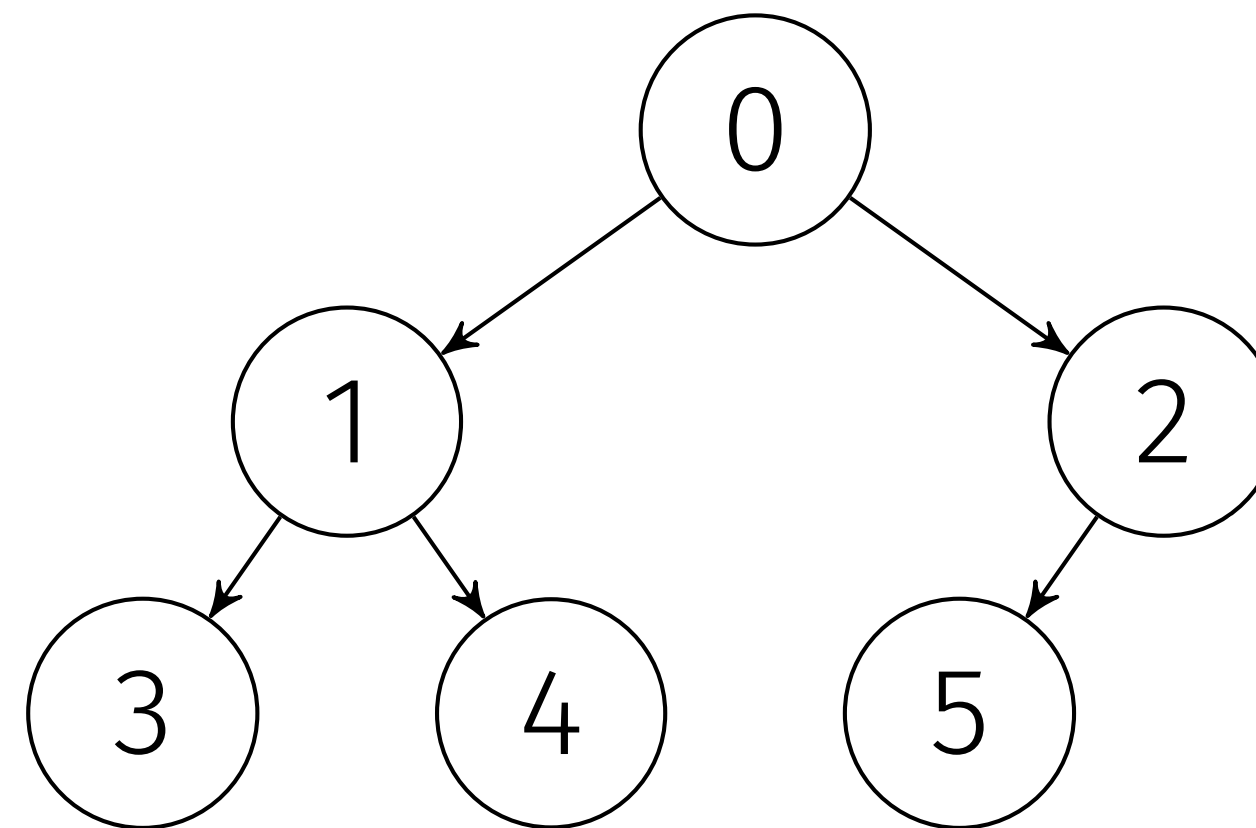
Voller Binärbaum

Ein Binärbaum ist ein voller Binärbaum, wenn jeder Knoten 0 oder 2 Kinder hat.



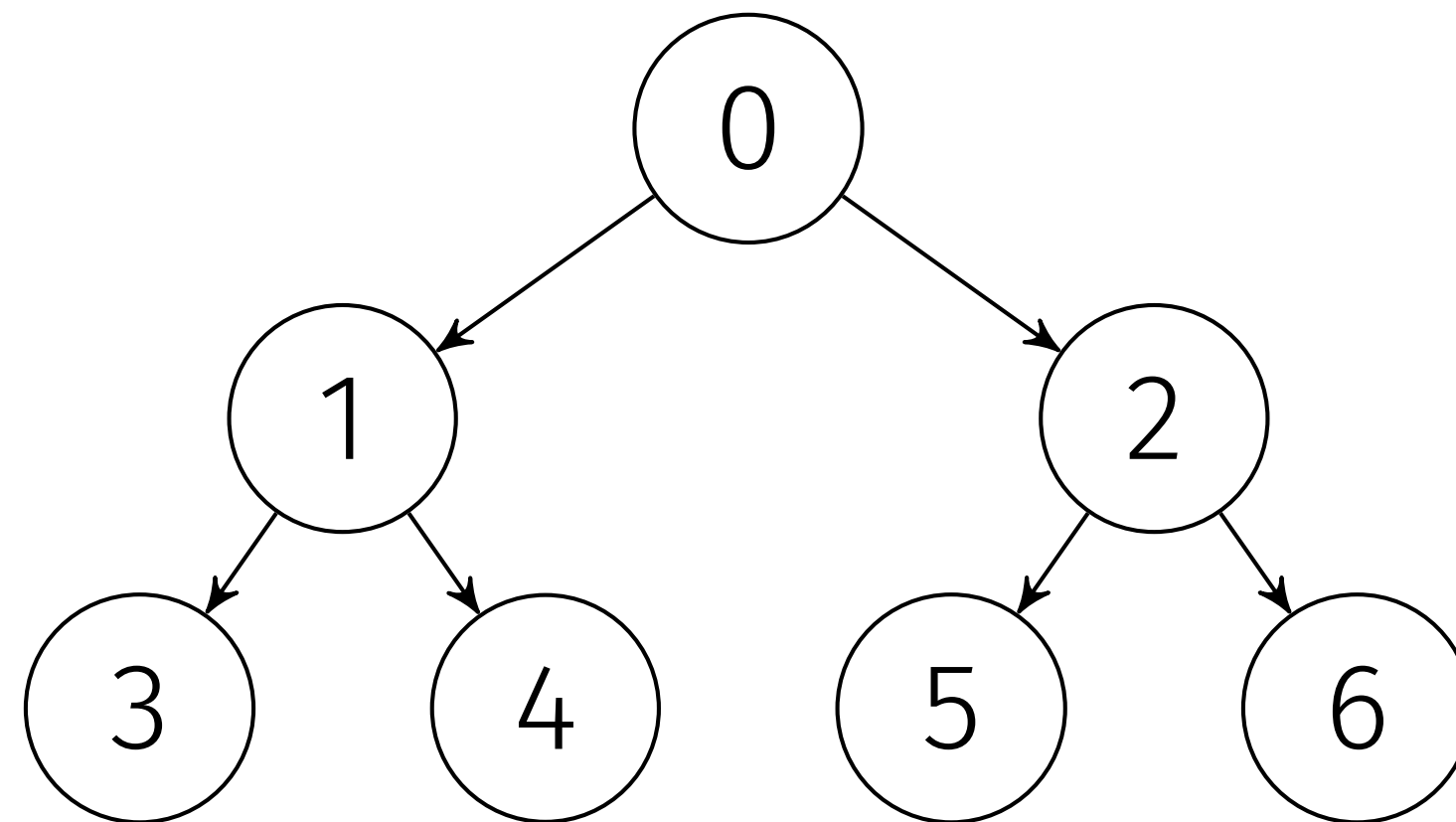
Vollständiger Binärbaum

Ein vollständiger Binärbaum ist ein Baum, in dem alle Ebenen vollständig gefüllt sind, ausser möglicherweise die letzte Ebene, die von links nach rechts gefüllt ist.



Perfekter Binärbaum

Ein perfekter Binärbaum ist ein Baum, bei dem alle inneren Knoten zwei Kinder haben und alle Blätter auf derselben Ebene sind.



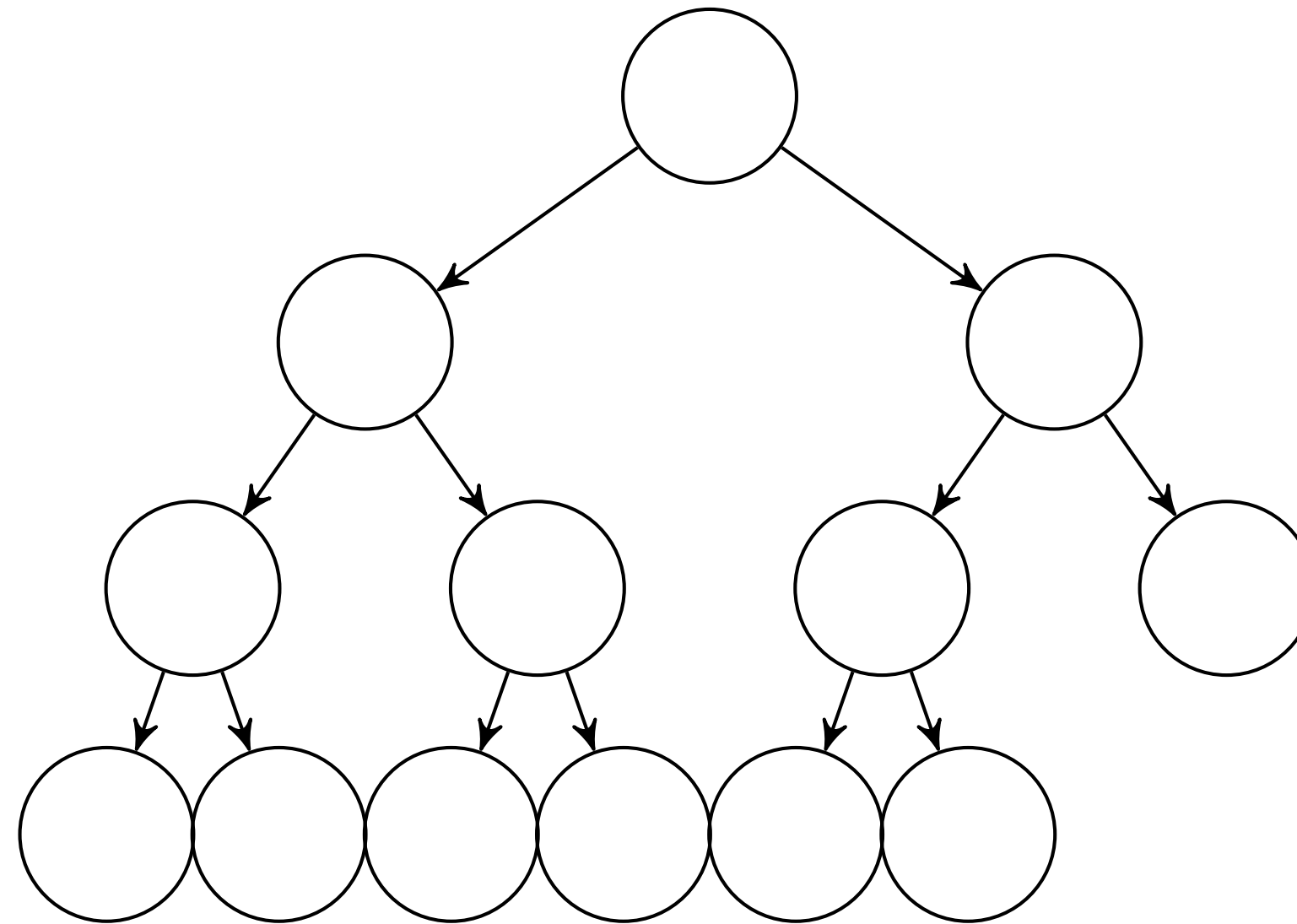
Binäre Bäume: Quiz

Wie viele Knoten enthält ein voller Binärbaum mit 6 Nicht-Blatt-Knoten höchstens:

- A. 6 Knoten
- B. 9 Knoten
- C. 11 Knoten
- D. 13 Knoten

Binäre Bäume: Quiz Antwort

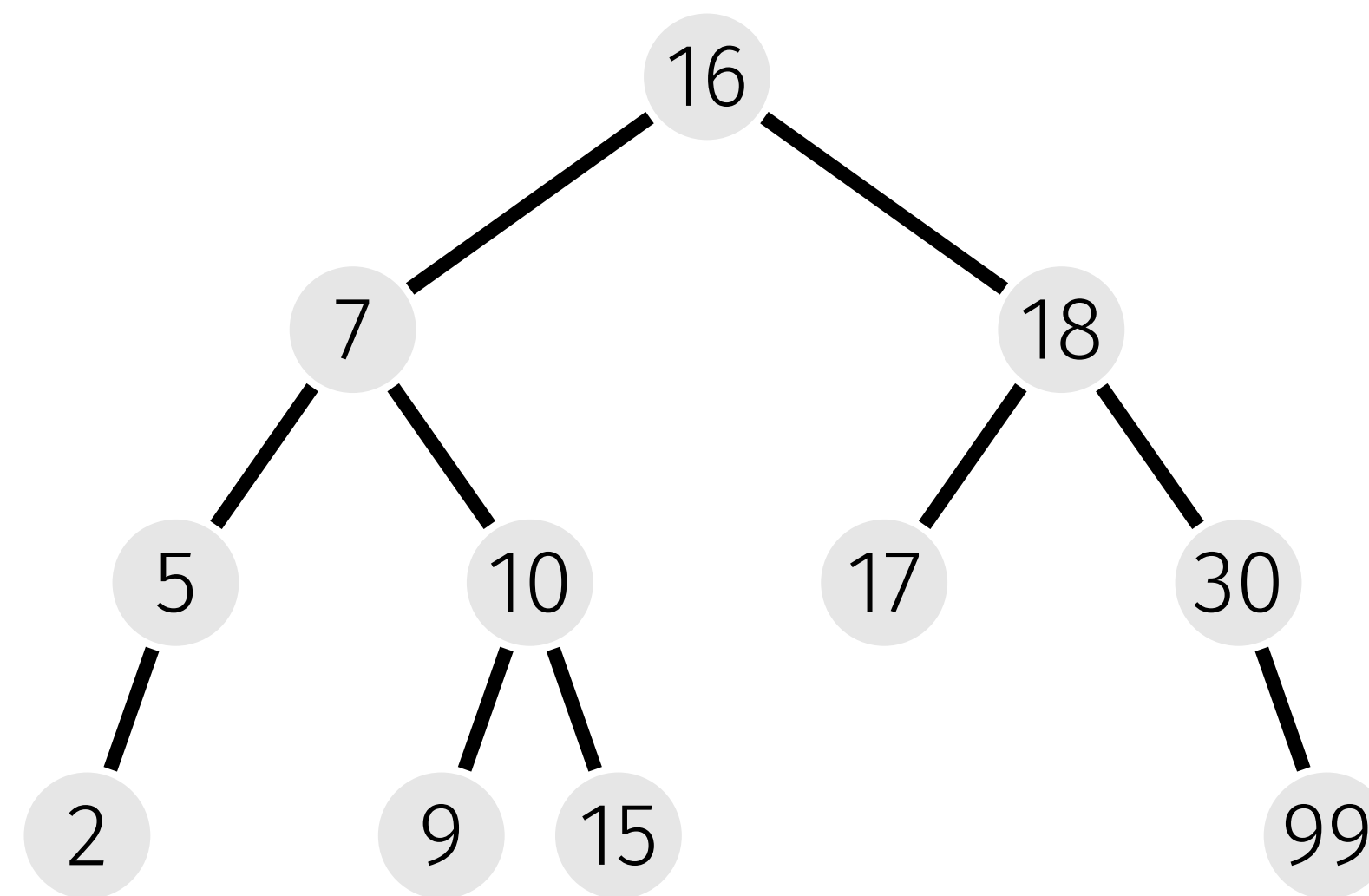
Ein voller Binärbaum mit n Nicht-Blatt-Knoten enthält $2n + 1$ Knoten. Also:
 $2 \times 6 + 1 = 13$.



Binäre Suchbäume

Ein *binärer Suchbaum* ist ein binärer Baum, der die **Suchbaumeigenschaft** erfüllt:

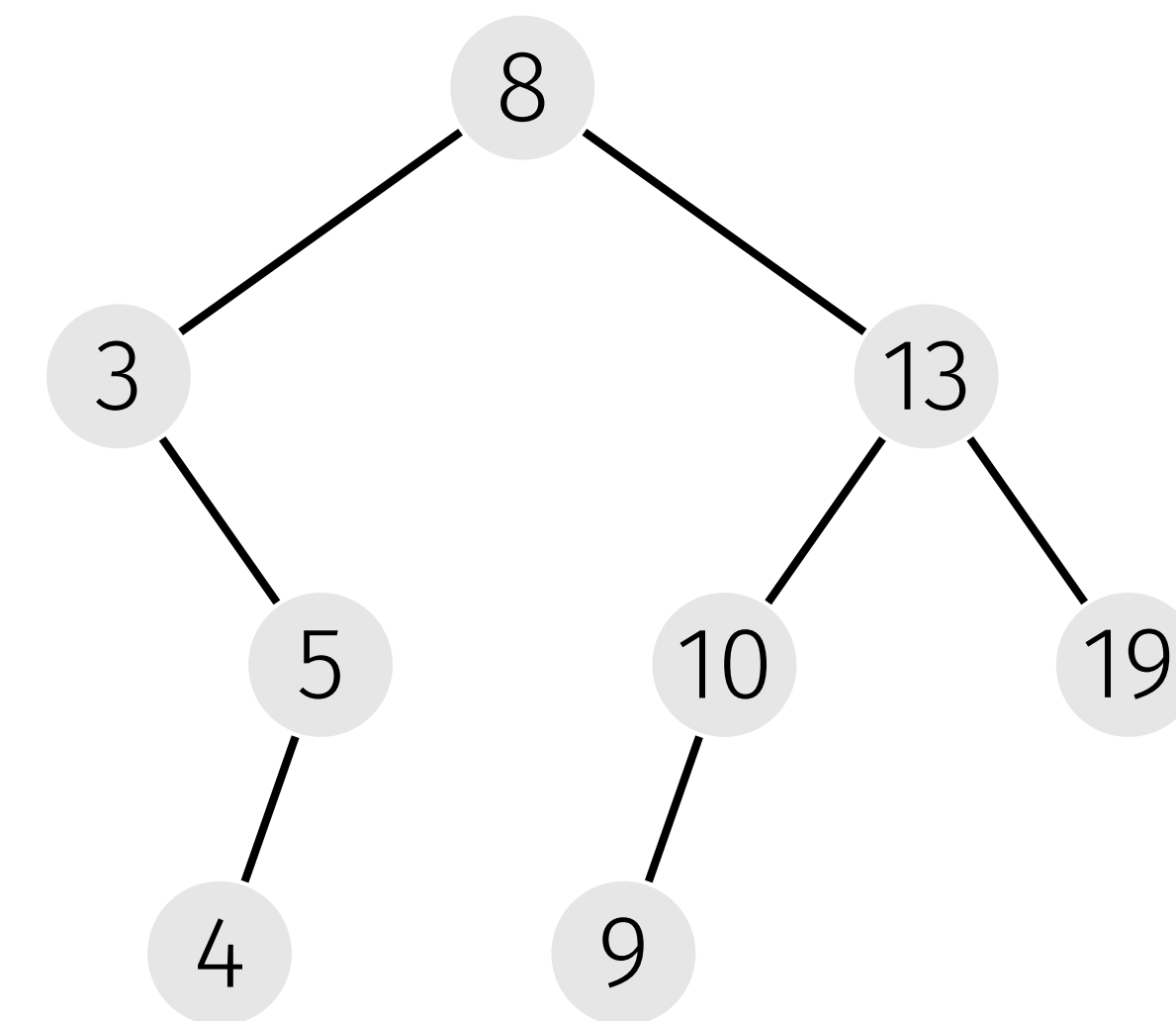
- Jeder Knoten v speichert einen Schlüssel
- Schlüssel im linken Teilbaum $v.\text{left}$ kleiner als $v.\text{key}$
- Schlüssel im rechten Teilbaum $v.\text{right}$ grösser als $v.\text{key}$



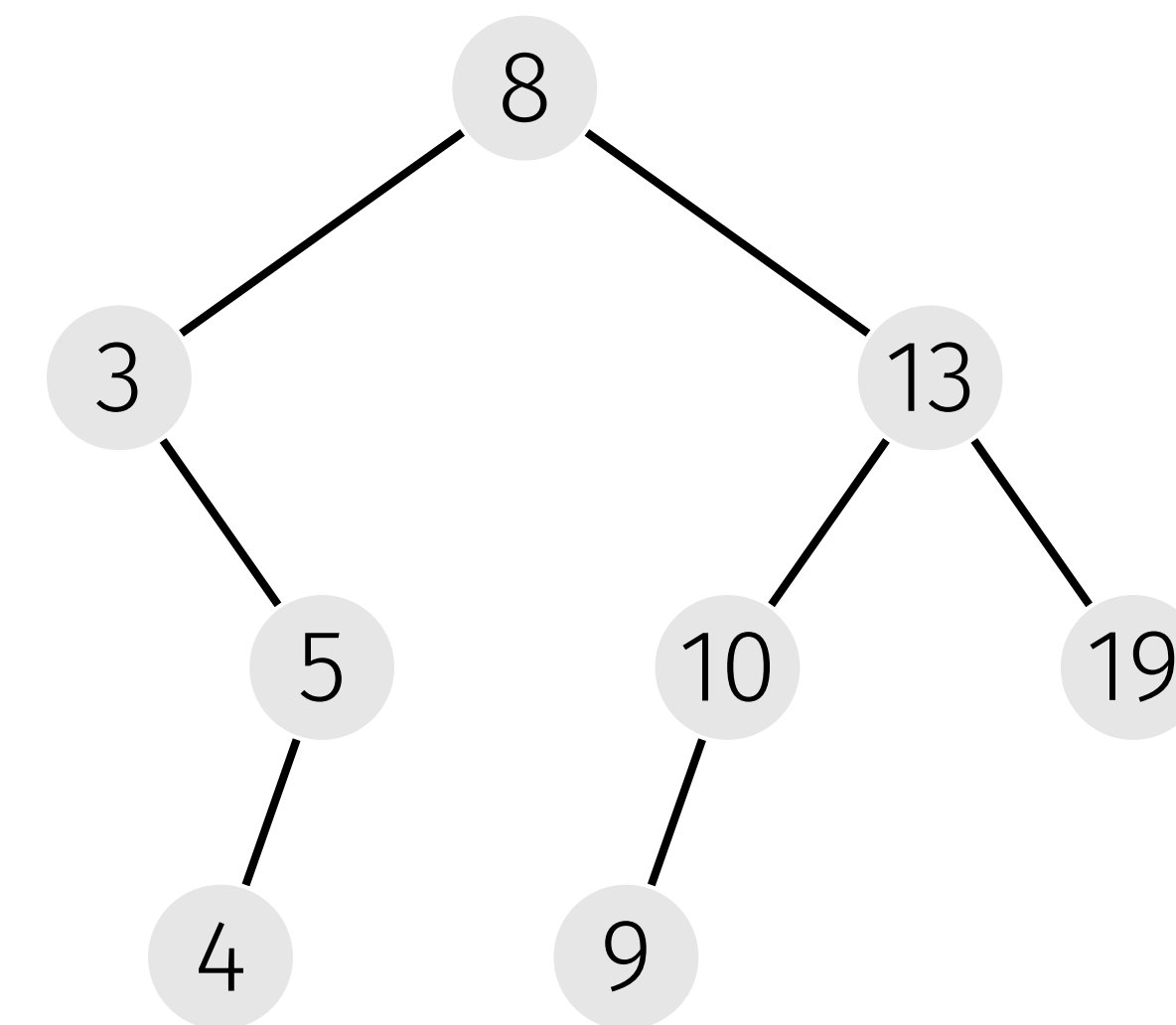
englischer Begriff viel gebräuchlicher!



- *Hauptreihenfolge* (*preorder*):
 v , dann $T_{\text{left}}(v)$, dann $T_{\text{right}}(v)$.



- *Hauptreihenfolge (preorder)*:
 v , dann $T_{\text{left}}(v)$, dann $T_{\text{right}}(v)$.
8, 3, 5, 4, 13, 10, 9, 19



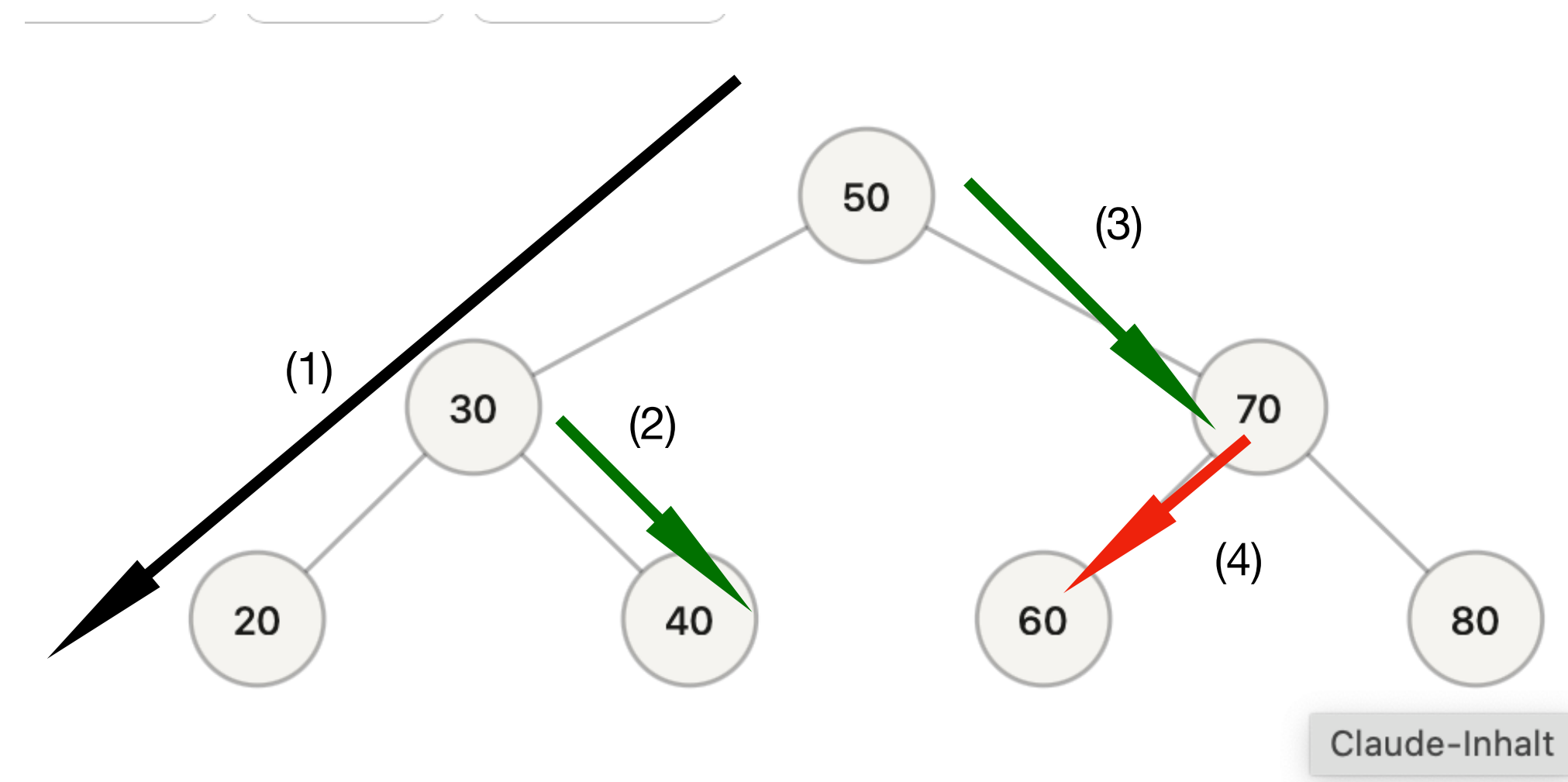
PreOrder

Zentrale Merkmale

Root, Left, Right

- Geht links durch, bis ganz nach unten
- dann wir von **unten nach oben in die rechten Teilbäume eingestiegen.**
- und bei jeder Gelegenheit links abgebogen.

PreOrder



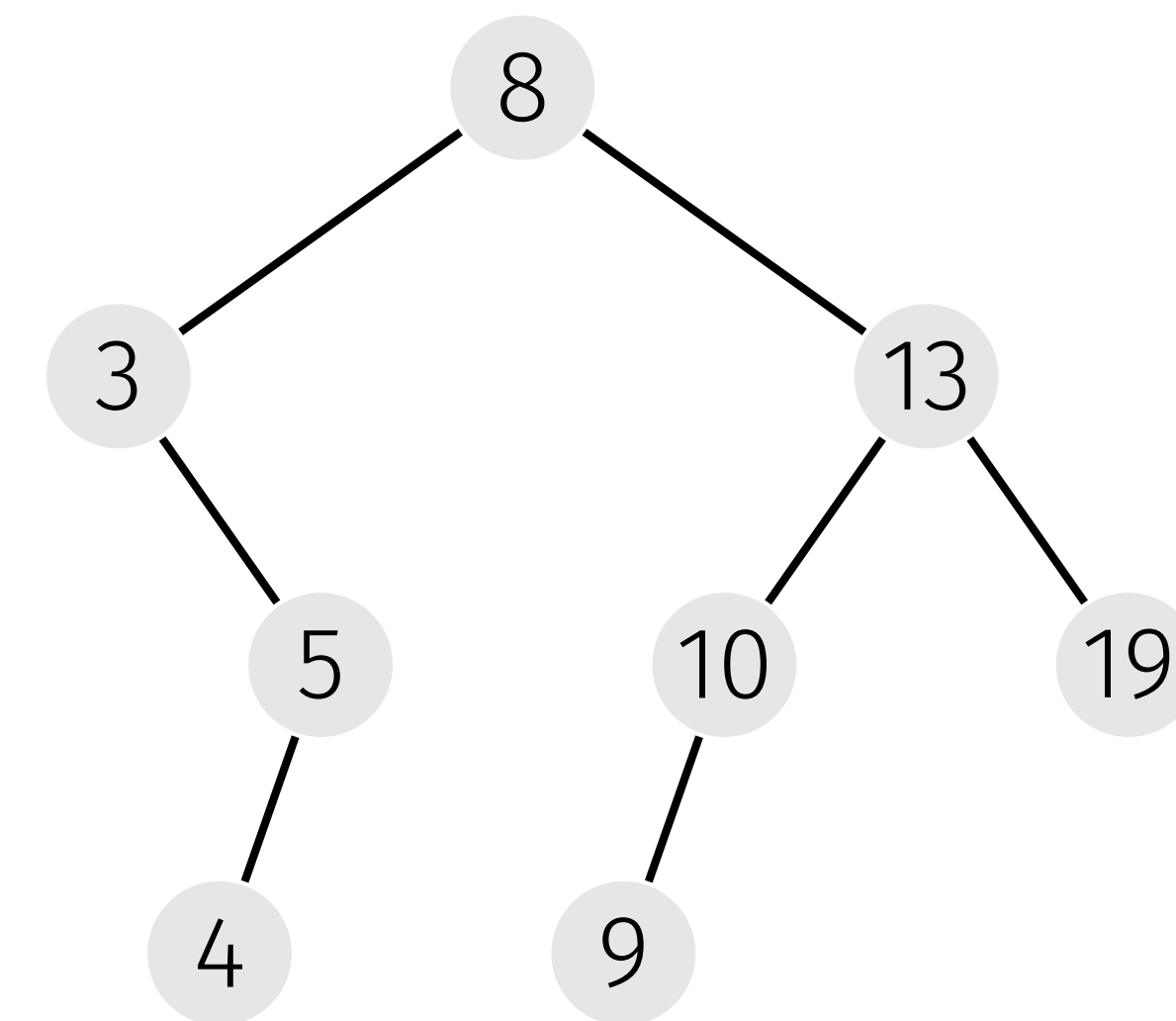
(1) 50,30,20

(2) 40

(2) 70

(4) 60

- *Hauptreihenfolge* (*preorder*):
 v , dann $T_{\text{left}}(v)$, dann $T_{\text{right}}(v)$.
8, 3, 5, 4, 13, 10, 9, 19
- *Nebenreihenfolge* (*postorder*):
 $T_{\text{left}}(v)$, dann $T_{\text{right}}(v)$, dann v .



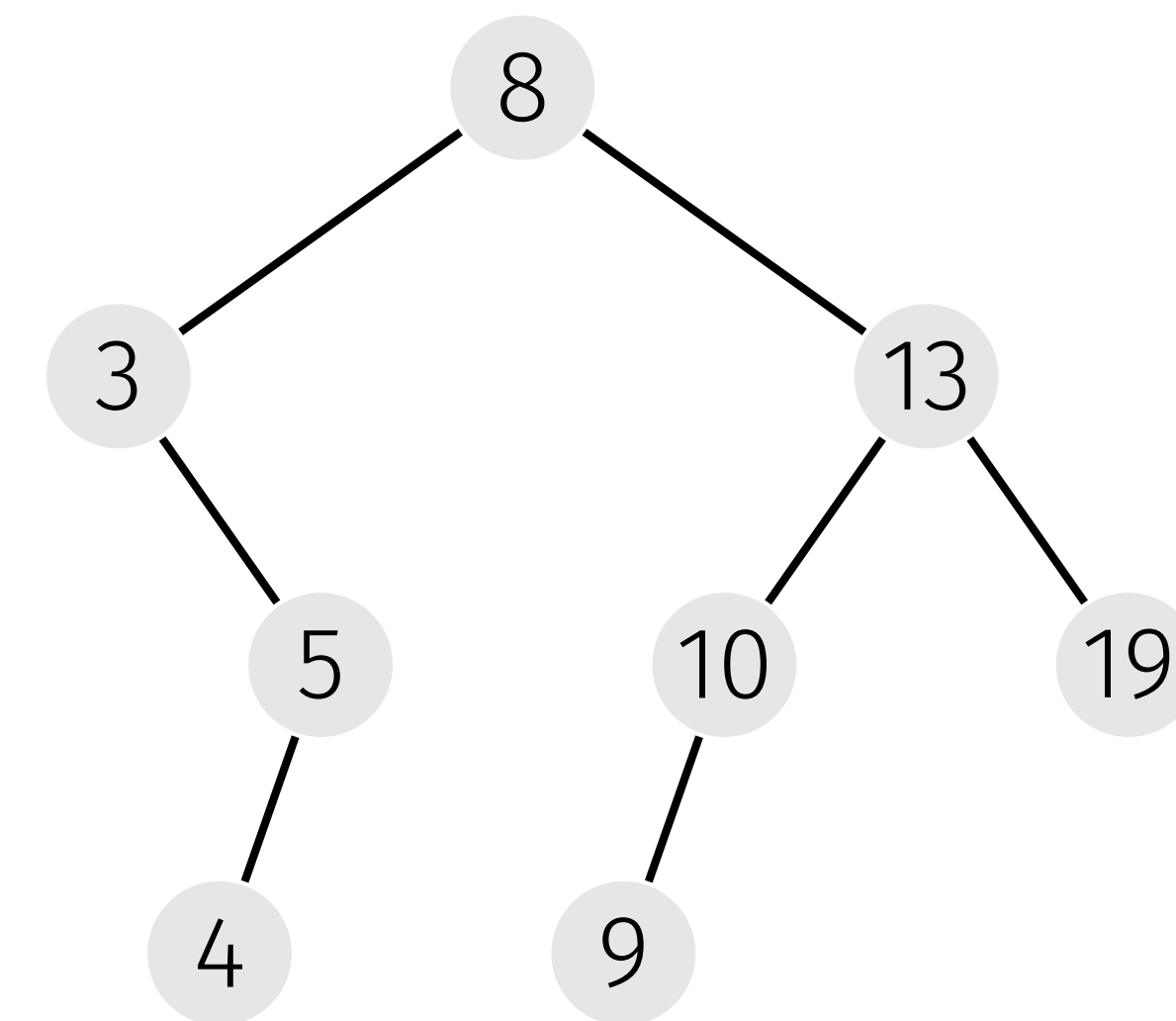
PostOrder

Zentrale Merkmale

Left, Right, Root

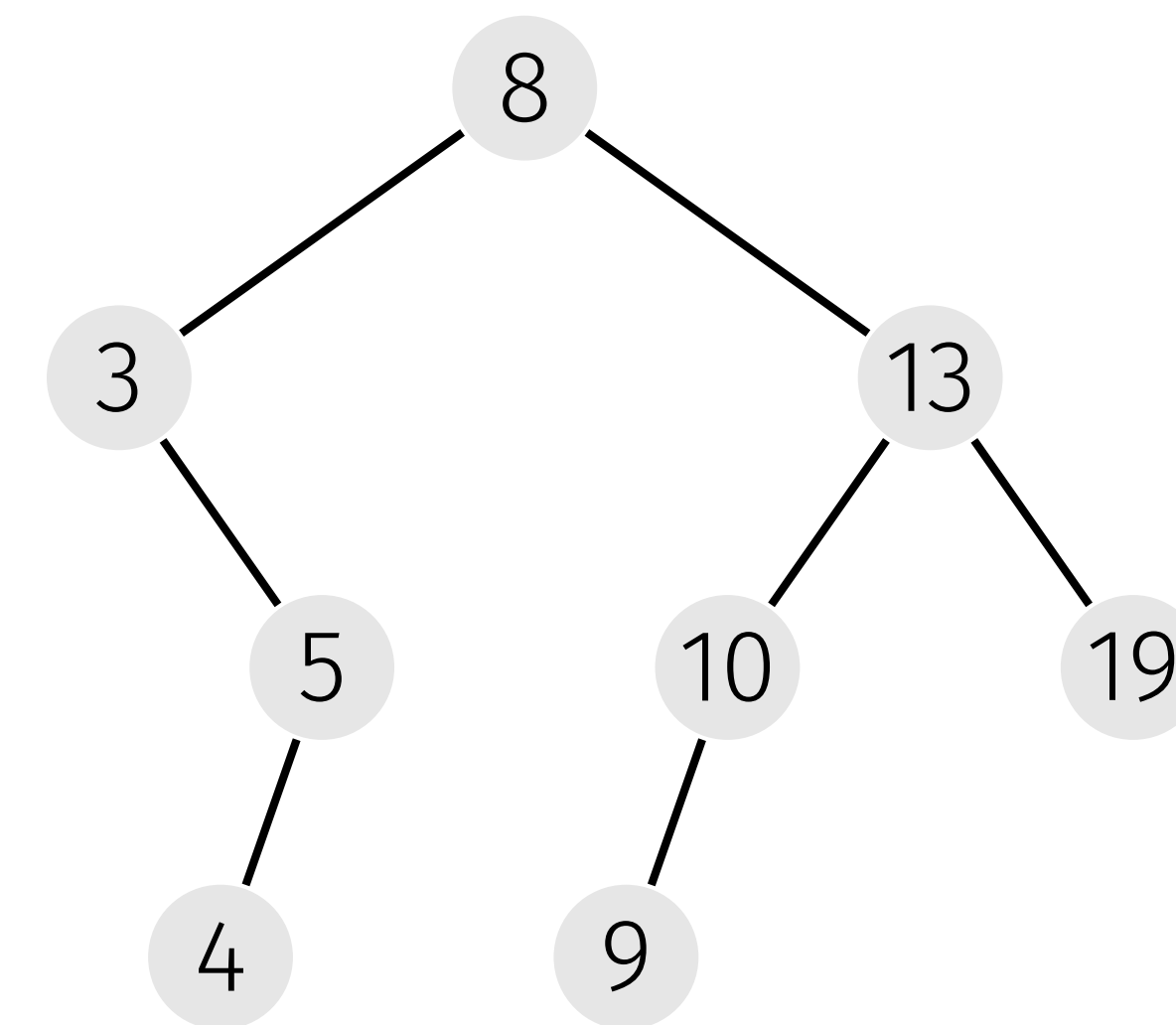
- (Starte ganz links unten)
- ein Key wird erst ausgegeben, wenn der gesamte Subbaum bereits davon ausgegeben ist.
- beachte die Reihenfolge, falls ein Subbaum noch nicht durchlaufen ist, wird zuerst links „abgebogen“, dann rechts..

- *Hauptreihenfolge* (*preorder*):
 v , dann $T_{\text{left}}(v)$, dann $T_{\text{right}}(v)$.
8, 3, 5, 4, 13, 10, 9, 19
- *Nebenreihenfolge* (*postorder*):
 $T_{\text{left}}(v)$, dann $T_{\text{right}}(v)$, dann v .
4, 5, 3, 9, 10, 19, 13, 8
- *Symmetrische Reihenfolge* (*inorder*):
 $T_{\text{left}}(v)$, dann v , dann $T_{\text{right}}(v)$.



Traversierungsarten

- *Hauptreihenfolge* (*preorder*):
 v , dann $T_{\text{left}}(v)$, dann $T_{\text{right}}(v)$.
8, 3, 5, 4, 13, 10, 9, 19
- *Nebenreihenfolge* (*postorder*):
 $T_{\text{left}}(v)$, dann $T_{\text{right}}(v)$, dann v .
4, 5, 3, 9, 10, 19, 13, 8
- *Symmetrische Reihenfolge* (*inorder*):
 $T_{\text{left}}(v)$, dann v , dann $T_{\text{right}}(v)$.
3, 4, 5, 8, 9, 10, 13, 19



InOrder

Zentrale Merkmale

Links, root, right

- (Starte ganz links unten)
- ein Key wird erst ausgegeben, wenn alle Elemente Links davon durchlaufen sind.

Merken: InOrder => aufsteigende Reihenfolge der Elemente

Bemerkung

Notation Reihenfolge:

- die weitverbreitete „Left,Right, Root“-Beschreibung ist als Funktion jedes einzelnen Keys (v) zu verstehen.

$T_{\text{left}}(v)$, dann $T_{\text{right}}(v)$, dann v

links

rechts

root

wichtig fürs korrekte Anwenden!

*gilt auch für die anderen Traversals

Traversierungsarten: Quiz

Zeichnen Sie jeweils einen binären Suchbaum, der die folgenden Traversierungen erzeugt. Ist der Baum eindeutig?

Symmetrische Reihenfolge (inorder) | 1 2 3 4 5 6 7 8

Geben Sie zu jeder Reihenfolge eine Zahlensequenz aus $\{1, \dots, 4\}$, die nicht aus einem gültigen binären Suchbaum stammen kann.

tipp: *Symmetrische Reihenfolge (inorder):*
 $T_{\text{left}}(v)$, dann v , dann $T_{\text{right}}(v)$.

Traversierungsarten: Quiz Antwort

- Jeder Suchbaum mit den Zahlen $\{1, \dots, 8\}$ ist gültig.
- Der Baum ist nicht eindeutig.
- Es gibt keinen Suchbaum, welcher nicht die aufsteigend sortierte Sequenz ausgeben würde. Gegenbeispiel: 1 2 4 3

Traversierungsarten: Quiz

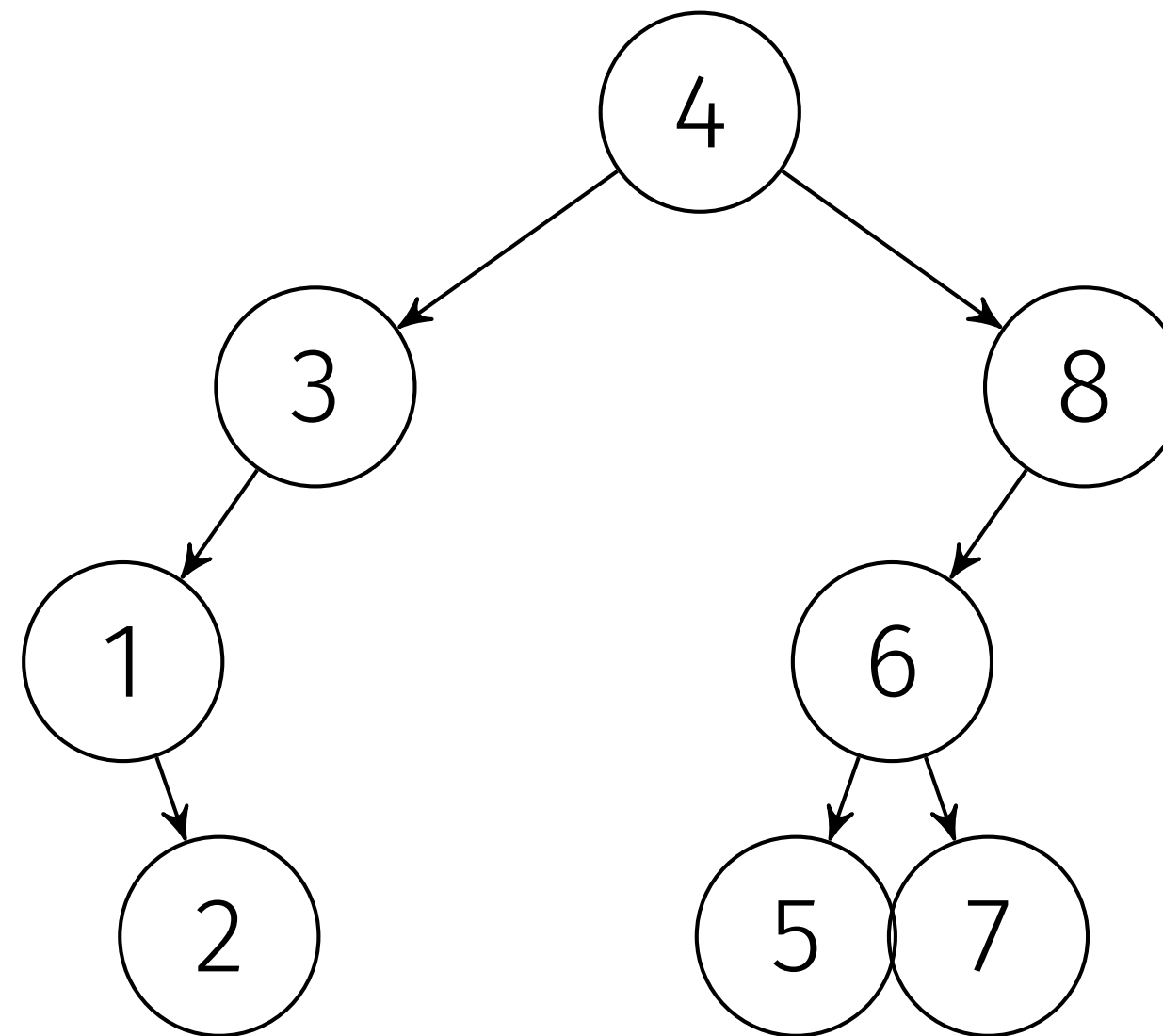
Zeichnen Sie jeweils einen binären Suchbaum, der die folgenden Traversierungen erzeugt. Ist der Baum eindeutig?

Hauptreihenfolge (preorder)	4 3 1 2 8 6 5 7
-----------------------------	-----------------

Hauptreihenfolge (preorder):
 v , dann $T_{\text{left}}(v)$, dann $T_{\text{right}}(v)$.

Traversierungsarten: Quiz Antwort

- Hauptreihenfolge (preorder): 4 3 1 2 8 6 5 7



- Der Baum ist eindeutig.

Weg zur Lösung

gegeben:

Hauptreihenfolge (preorder):

v , dann $T_{\text{left}}(v)$, dann $T_{\text{right}}(v)$. : (preorder): 4 3 1 2 8 6 5 7

- 4 als erstes element muss eine root sein.
- - 3 < 4 => links von 4; gleiches für 1 < 3...
- Suchbaum: 2 darf nicht Links von der 1 stehen => muss rechts von der 1 sein.
- Suchbaum: 8 > 4 => rechts von der 4.

Traversierungsarten: Quiz

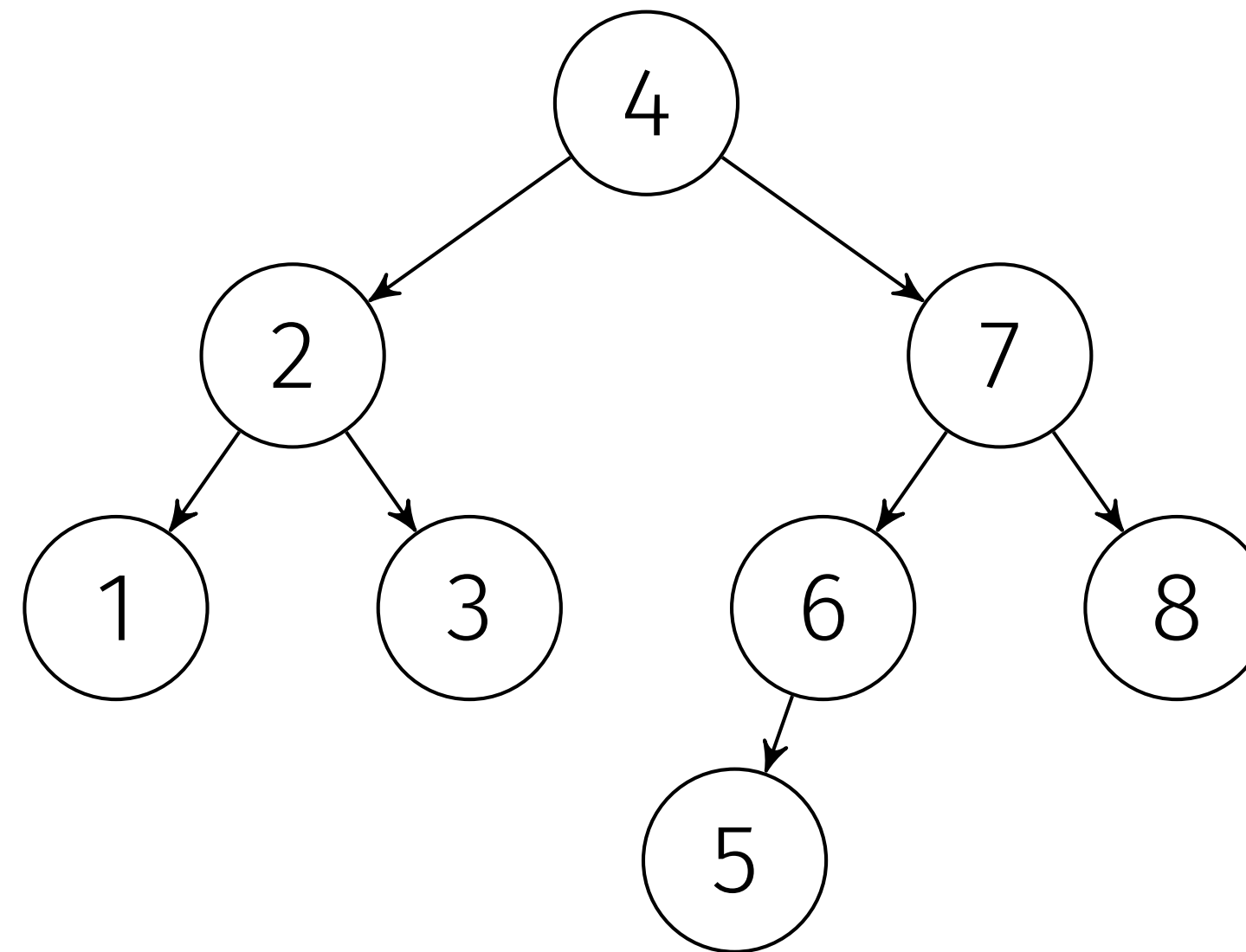
Zeichnen Sie jeweils einen binären Suchbaum, der die folgenden Traversierungen erzeugt. Ist der Baum eindeutig?

Nebenreihenfolge (postorder)	1 3 2 5 6 8 7 4
------------------------------	-----------------

Nebenreihenfolge (postorder):
 $T_{\text{left}}(v)$, dann $T_{\text{right}}(v)$, dann v .

Traversierungsarten: Quiz Antwort

- Nebenreihenfolge (postorder): 1 3 2 5 6 8 7 4



- Der Baum ist eindeutig.
- Konstruktion hier: <https://www.techiedelight.com/build-binary-search-tree-from-postorder-sequence/>, Ähnliches Argument wie vorher, nur von hinten nach vorne. Gegenbeispiel 4 2 1 3

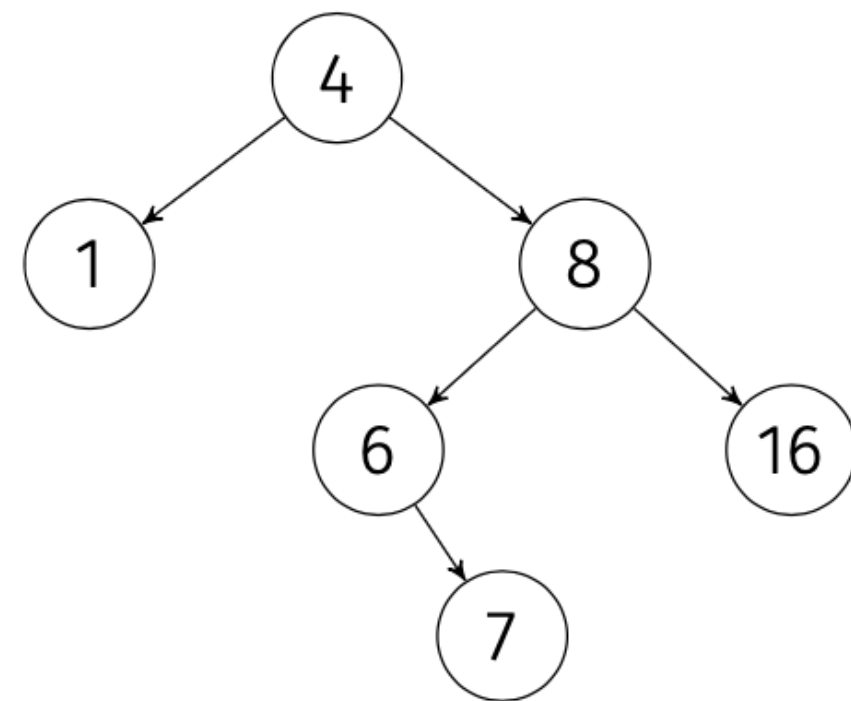
Tree Bauen & Einsetzen:

beide genau gleich!

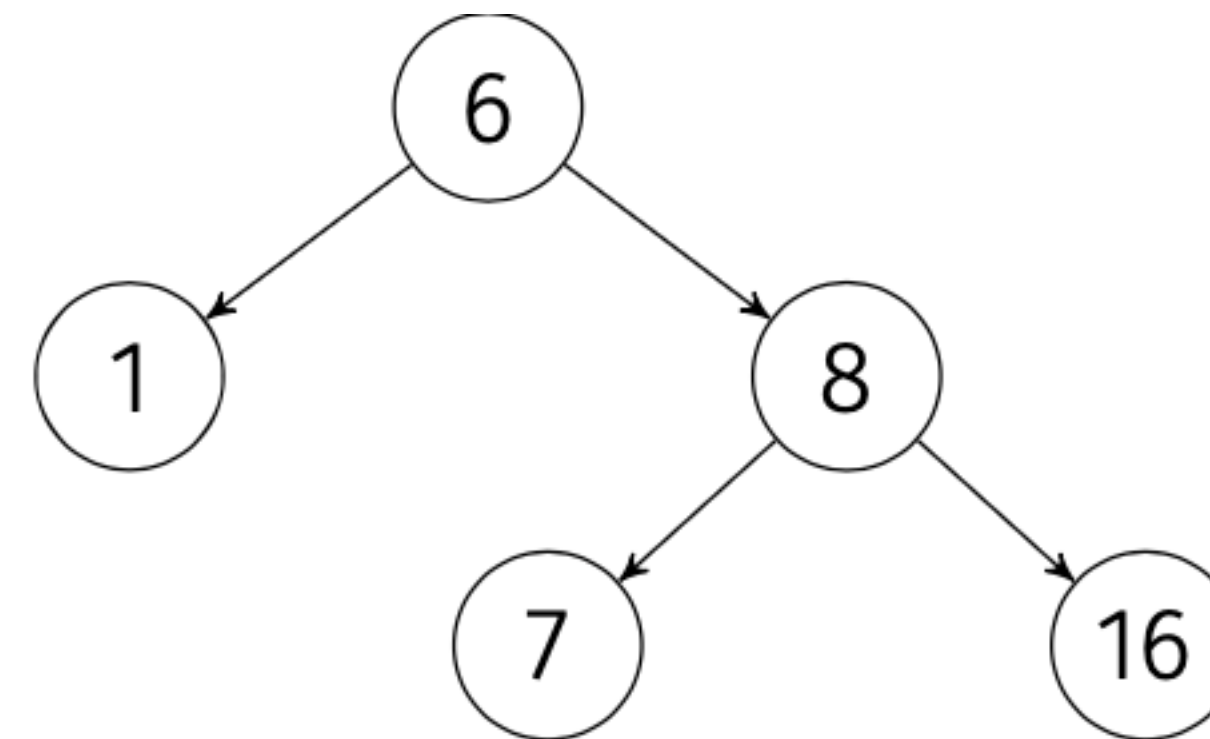
- Mit dem einzusetzenden wert durch den Baum navigieren:
 - Falls kleiner als ein Knoten links davon, ansonsten rechts.
 - Bis zum Ende & platzieren.

Elemente aus BST löschen

- zu löschendes Element mit InOrder-Nachfolger swappen.
(inOrder-Nachfolger = nächst grösseres Element)
- element Löschen.
- Falls der ursprüngliche InOrder-Nachfolger ein child hatte: dieses nachrücken lassen.

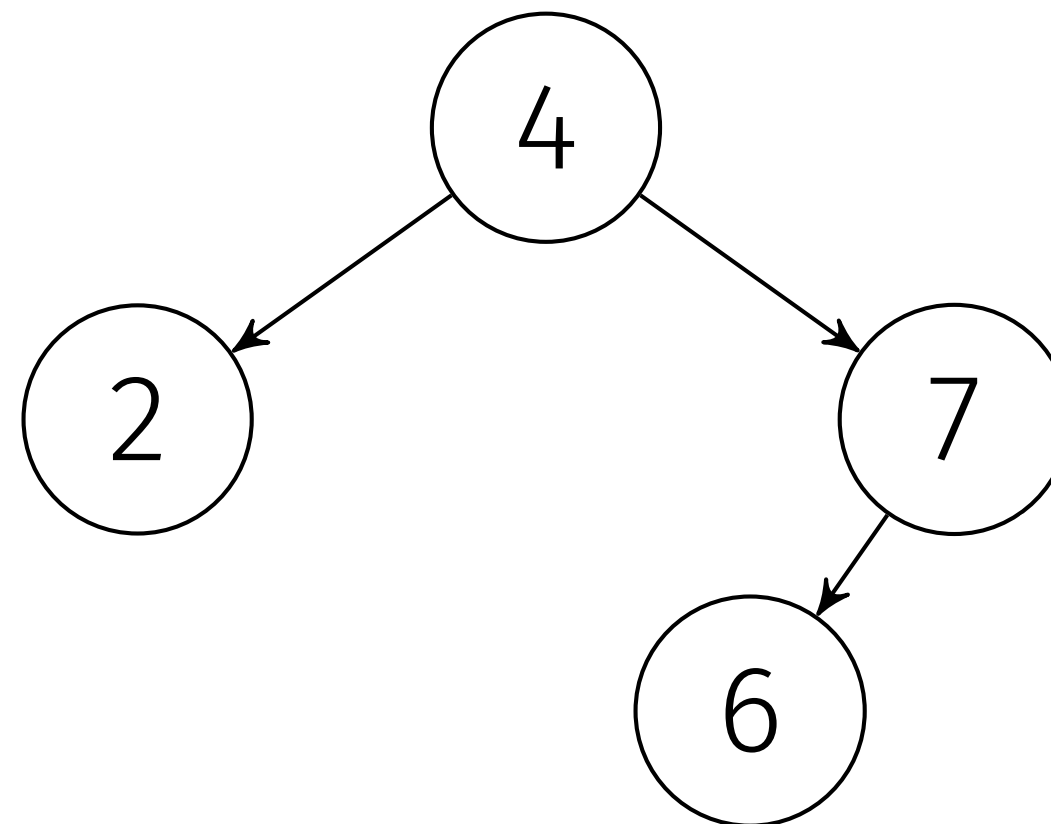


4 löschen



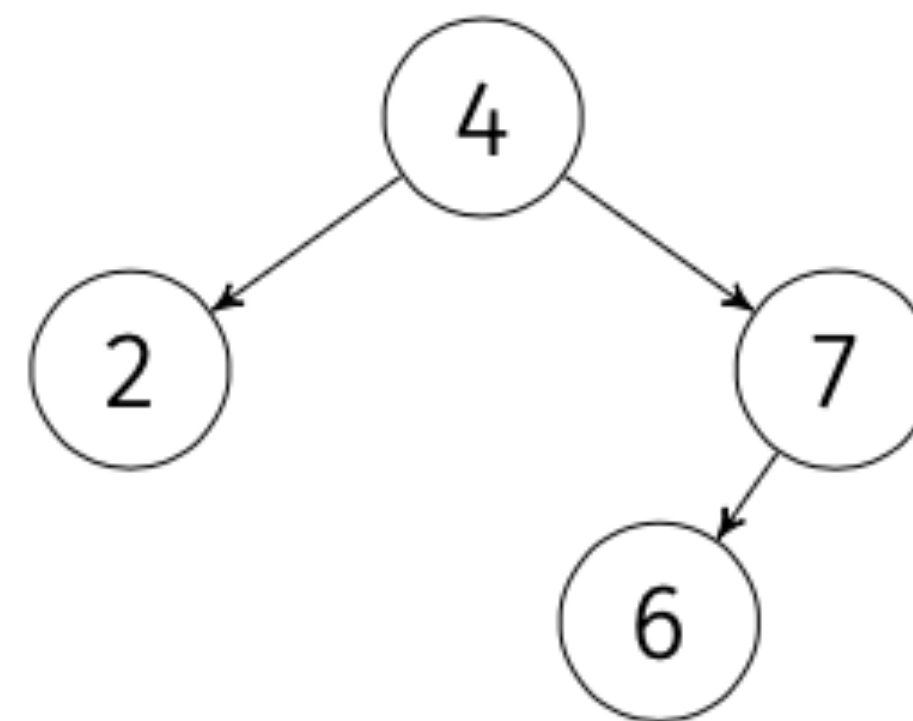
Schlüssel Löschen: Quiz

Wenn Sie zwei Schlüssel aus einem Binären Suchbaum löschen wollen, spielt es dann eine Rolle, in welcher Reihenfolge Sie dies tun? Mit anderen Worten, ist das Löschen kommutativ?

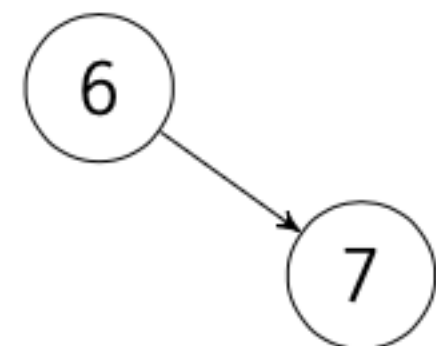


Operationen im BST nicht kommutativ

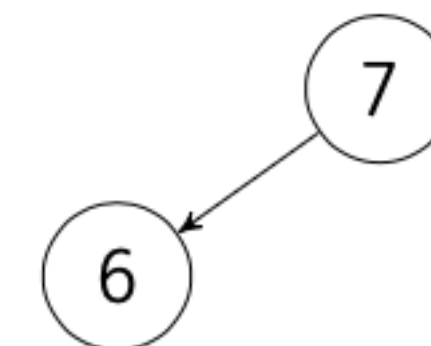
Löschen von 4,2 vs 2,4



original



4,2



2,4

Heaps

Ein *Heap* ist eine baumbasierte Datenstruktur wobei:

- Der Baum ein **vollständiger Binärbaum** ist.
- Ein Heap mit **N** Knoten eine Höhe von **$\log N$** hat (Eigenschaft eines vollständigen Binärbaums).
- Es ist für die schnelle Extraktion von Minimum oder Maximum und für das Sortieren optimiert.

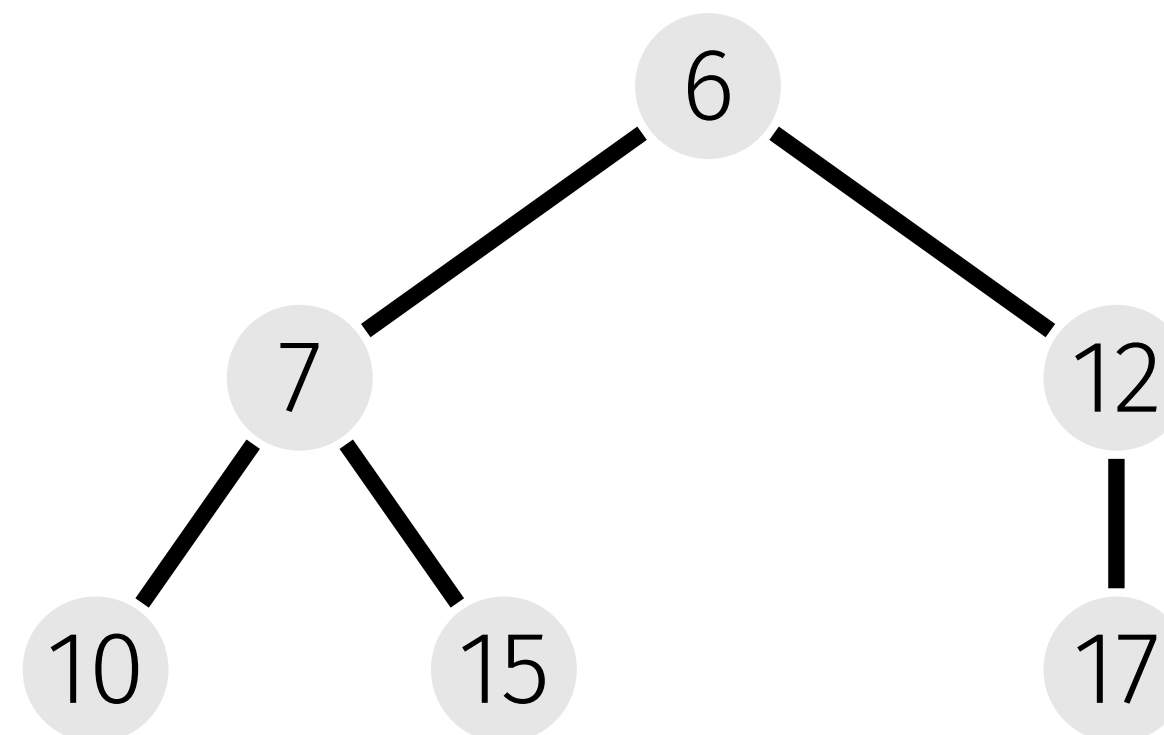
Es gibt zwei Arten von Heap-Implementierungen:

- Min-Heaps
- Max-Heaps

Min-Heaps

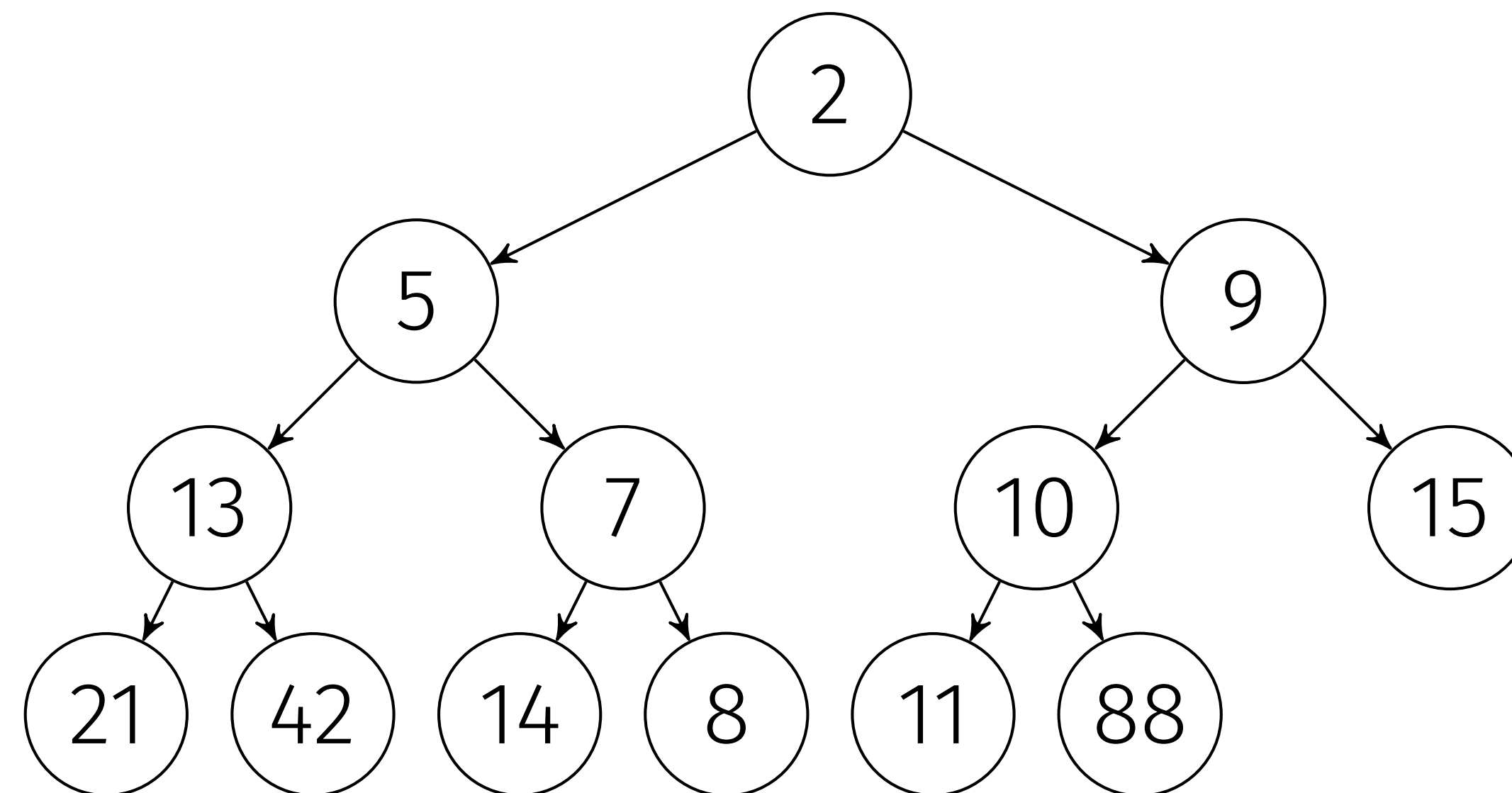
Ein *Min-Heap* ist ein binärer Baum wo:

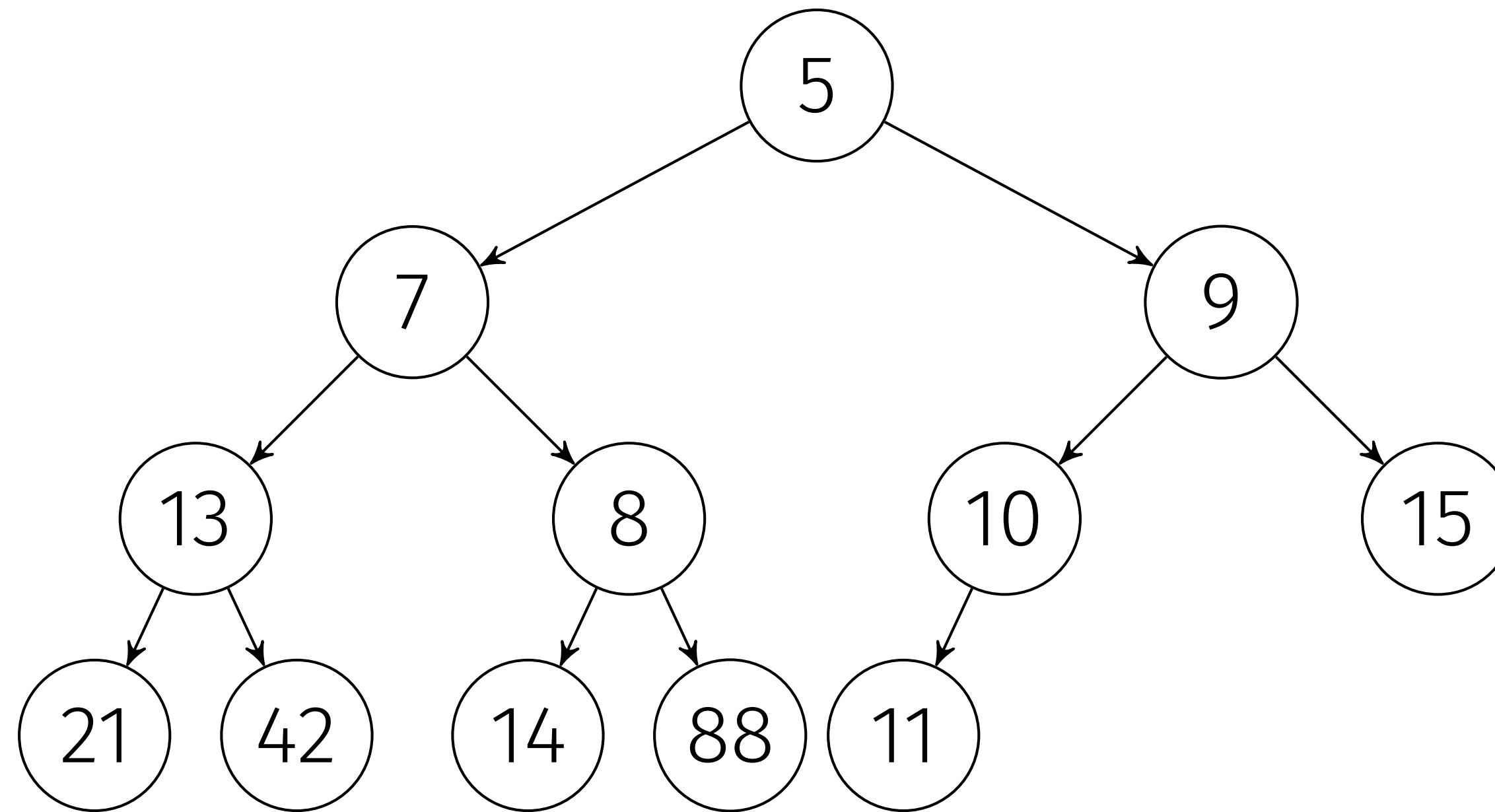
- Der am Wurzelknoten vorhandene Schlüssel ist immer **kleiner oder gleich** den Schlüsseln aller seiner Kinder.
- Dieselbe Eigenschaft muss für alle Teilbäume in diesem Binärbaum rekursiv wahr sein.
- Das **kleinste** Schlüsselement ist daher immer an der Wurzel vorhanden.



Min-Heaps: Quiz

Führen Sie auf folgendem Min-Heap eine Extract-Min (entferne den kleinsten Schlüssel) Operation aus, wie in der Vorlesung vorgestellt, einschliesslich der Wiederherstellung der Heap-Bedingung. Wie sieht der Heap nach der Operation aus?





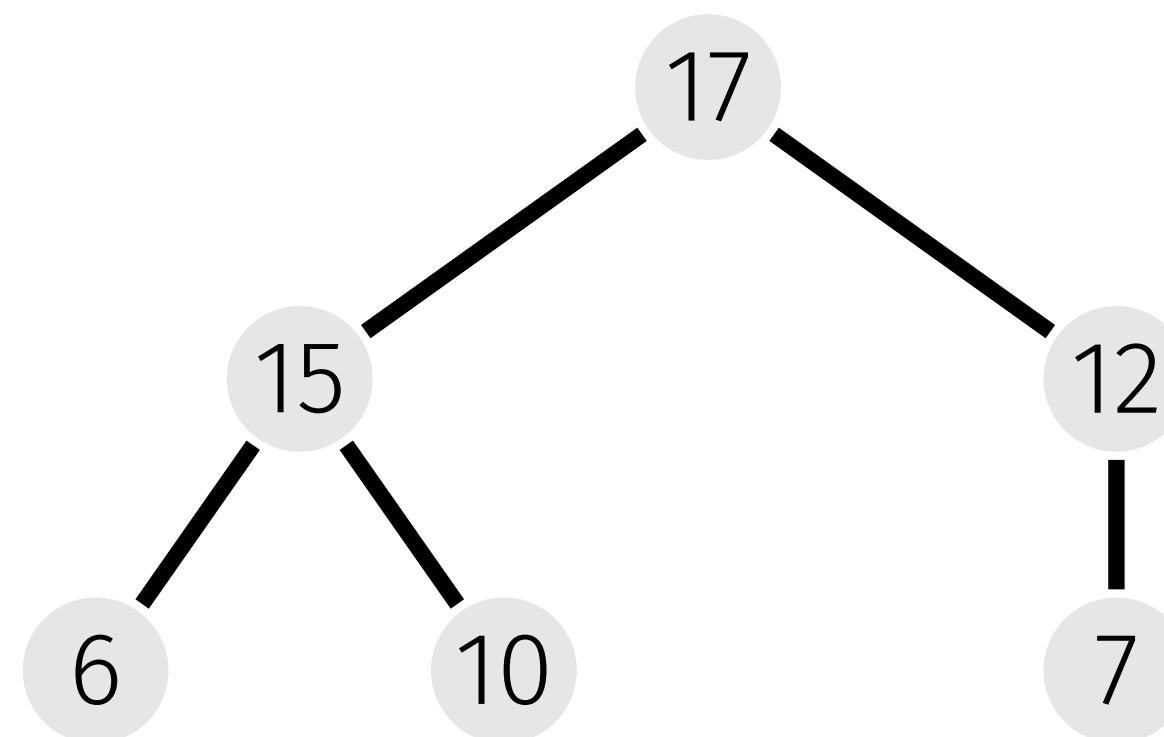
Vorgehen:

- kleinstes element entfernen (die spitze)
- element ganz rechts in der untersten ebene an die spitze setzen (88)
- Sift-Down: die 88 mit dem kleineren element jeweils tauschen bis heap bedingungen hergestellt.

Max-Heaps

Ein *Max-Heap* ist ein binärer Baum wo:

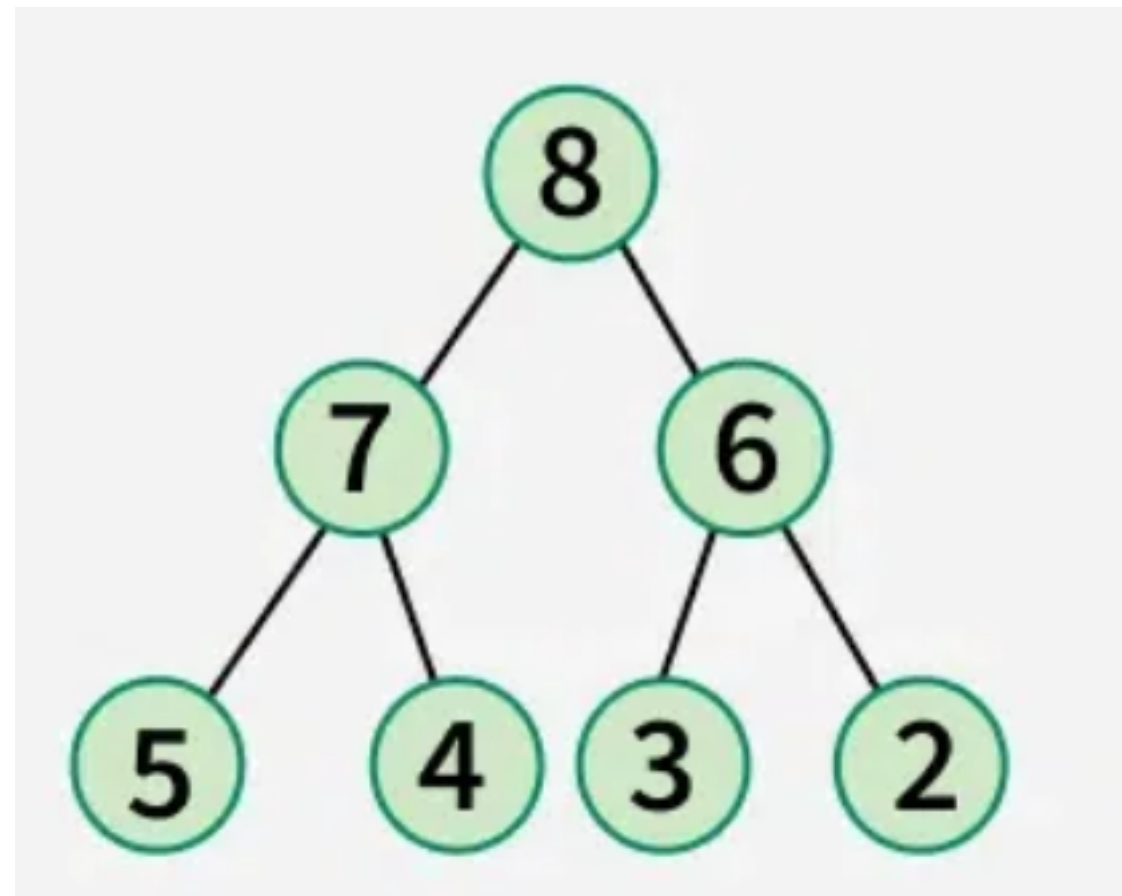
- Der am Wurzelknoten vorhandene Schlüssel ist immer **größer oder gleich** den Schlüsseln aller seiner Kinder.
- Dieselbe Eigenschaft muss für alle Teilbäume in diesem Binärbaum rekursiv wahr sein.
- Das **größte** Schlüsselement ist daher immer an der Wurzel vorhanden.



Heaps speichern

wichtige Eigenschaften!

- wir können Heaps bequem als eine Liste speichern, denn der aufbau ist klar geregelt.



= [8, 7, 6, 5, 4, 3, 2]

wobei wir für ein Element mit index i wissen:

linkes kind hat index= $2*i + 1$

rechtes kind index $2*i+2$.

parent $(i-1//2)$ ($//$ ist abrunden bzw. teilen ohne rest.)

Schlüssel Einfügen

Binärer Suchbaum

- Nach Schlüssel suchen.
- Bei erreichtem leeren Blatt (`null`) einfügen.

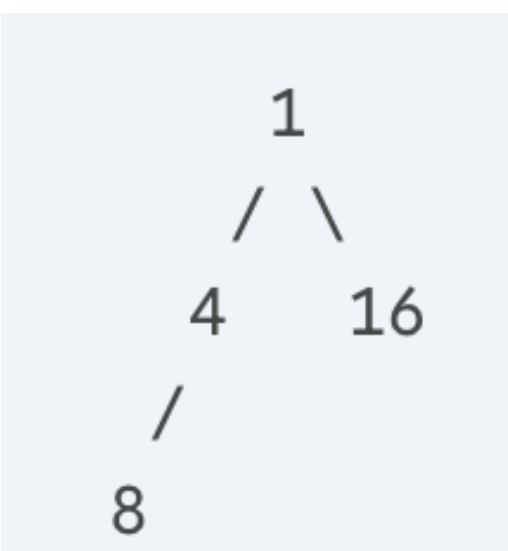
Min-Heap

- Zuhinterst im Array einfügen.
- Heap-Bedingung wiederherstellen: `siftUp` (Aufsteigen lassen).

Beispiel Heap bauen

Einfügen von 4, 8, 16, 1, 6, 7 in einen Min-Heap

- starte mit 4, ist root (einziges element) [4]
- 8 unten platzieren, $4 < 8 \Rightarrow$ zulässig. [4,8]
- 16 neben der 8 platzieren. $4 < 16 \Rightarrow$ zulässig. [4,8,16]
- 1 einfügen. Die 1 kommt unter die 8. ([4,8,16,1]) ; $8 > 1 \Rightarrow$ **nicht zulässig.**
 - 8 mit der eins tauschen $\Rightarrow$ [4,1,16,8]; $4 > 1 \Rightarrow$ **nicht zulässig.**
 - 4 mit der eins tauschen. $\Rightarrow$ [1,4,16,8]
- usw.



Element/Schlüssel löschen in Min-Heap

Vorgehen

- kleinstes element entfernen (die spitze)
- element ganz rechts in der untersten ebene an die spitze setzen. Dies ist das **letzte element in der Liste**.
- Sift-Down: das neue element an der spitze. mit dem kleineren element jeweils tauschen bis heap bedingungen hergestellt.

Komplexitätsanalyse von Min-Heap und Max-Heap

- Erhalten des größten oder kleinsten Elements: **$O(1)$**
- Element in Max-Heap oder Min-Heap einfügen: **$O(\log N)$**
- Größte- oder Kleinste-Element entfernen: **$O(\log N)$**

warum?

Exercise 7: Trees, Fällig 20,4

Bemerkungen& Tipps

- **Binäre Suchbäume:** theorie stumpf anwenden, am besten mit iPad/papier lösen.
- **Binärer Suchbaum:** **Wichtig für die Prüfung!** Typische implementierung von **OOP**
- **Heaps konkatenieren:** gedankenübung.
- **Heapsort:** -

Bis nächste Woche

Anonymes Feedback Formular

